ASP.NET Core Project Structure

What each file does, who reads it, and when

Create the project

```
dotnet new web -n ProjectStructure
cd ProjectStructure
code .
```

- `dotnet new` creates a project from a ready-made **template**.
- `web` is the **smallest** web template: an empty project that answers "Hello World!".
- Other templates (like `webapi`) come with many files we do not know yet.
- We start empty so we can understand **every** file.

> **bin/** and **obj/** folders appear after the first build. They are build output, so we never edit them.

THE WHOLE PROJECT: 5 FILES

```
ProjectStructure/
├── Properties/
│   └── launchSettings.json
├── appsettings.json
├── appsettings.Development.json
├── Program.cs
└── ProjectStructure.csproj
```

Group	Files	Question it answers
Project definition	`.csproj`	What is this project and what does it need?
Code	`Program.cs`	What does the app do?
Settings and launch	`appsettings*.json`, `launchSettings.json`	How is it configured and launched?

ProjectStructure.csproj

The project definition file: the project's ID card

1 What is it?

The file that defines the project. The extension `csproj` means **C# Project**, and it is written in XML.

2 Why does it exist?

"Project" in .NET means this file. Without it, the `dotnet` command does not know the folder is a project. Every project has exactly one `.csproj`.

3 Who reads it, and when?

The **dotnet** tool on `dotnet build` and `dotnet run`, and the editor when it opens the project. It is used to **build** the app, not while the app runs.

4 Do I edit it?

Rarely by hand. When you run `dotnet add package`, the command edits it for you.

INSIDE

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Setting	Meaning
`Sdk="...Sdk.Web"`	The project type. This is what makes it a **web** project, so ASP.NET Core libraries are available without adding them. With `Microsoft.NET.Sdk` it would be a plain console app.
`TargetFramework`	The .NET version the app runs on.
`Nullable`	Warnings that protect you from `null` mistakes.
`ImplicitUsings`	Common `using` lines are added for you. That is why `Program.cs` has no `using`.

> There are no **packages** because the project is simple. When we add one (like Entity Framework), a `PackageReference` line appears here.

Program.cs

The most important file: where the application starts

1 What is it?

The **entry point** of the application. The first line of this file is the first line that runs.

2 Why does it exist?

Every program needs a starting point. In older projects it was a `Main` method plus a `Startup.cs` file. Now everything is in one file thanks to C# **top-level statements**, so we do not write `Main`.

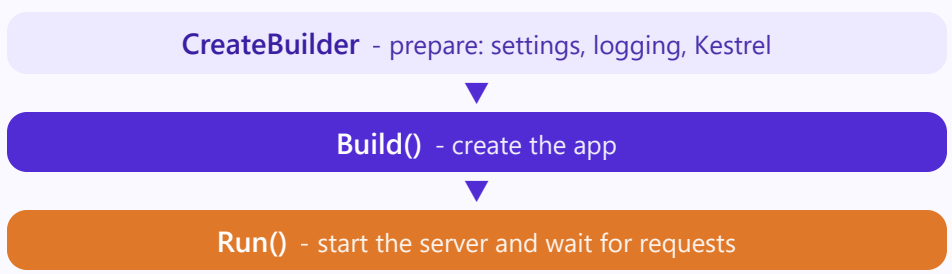
3 Who reads it, and when?

It runs **once** when the app starts, to build the app. Then the app keeps running (`app.Run()`) and waits for requests.

4 Do I edit it?

Yes, always. This is where we build the app.

WHAT THE 3 LINES DO



INSIDE: ONLY 3 LINES

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Run();
```

Line	What it does
`CreateBuilder(args)`	Creates the **builder**, a tool that prepares the app. It reads the settings (`appsettings.json`) and prepares logging and the Kestrel web server. `args` are the command-line arguments.
`builder.Build()`	Turns what the builder prepared into a ready **app**.
`app.Run()`	Starts the server and keeps waiting for requests until you stop it. No code after it runs.

Program.cs: three stages

Where each kind of code goes

CreateBuilder → **Build()** → Run()

A After CreateBuilder, before Build()

The `builder` (a **WebApplicationBuilder**) is where we **configure the project** and **register services in the IoC Container**.

- **Settings:** `builder.Configuration`, `builder.Logging`, `builder.Environment`
- **Services:** `builder.Services.AddXxx()`. The IoC Container holds the classes the app needs and hands them to whoever asks.

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();

var app = builder.Build();
```

> At **Build()** the app is created and the container is **locked**. You cannot add new services after it.

B After Build(), before Run()

Here we decide **how each HTTP request is handled**, using a chain of **middlewares**. A request passes through them one after another until it reaches the code that answers it.

- `app.UseXxx()` adds a **middleware**.
- `app.MapXxx()` registers a **route**.

```
var app = builder.Build();

app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/", () => "Hello World!");
app.MapControllers();

app.Run();
```

Before Build()	After Build(), before Run()
Configuration, Logging	`app.Use...` (middlewares)
`builder.Services.Add...`	`app.Map...` (routes)
Runs once at startup	Every request passes through

appsettings.json

and `appsettings.Development.json`: the application settings

1 What are they?

The settings of the application, written in JSON.

2 Why do they exist?

To keep settings that change (like a database address) **out of the code**, so you can change them without recompiling.

3 Who reads them, and when?

`CreateBuilder` reads them **automatically** at startup (the first line of `Program.cs`), so we write no code to read them.

- `appsettings.json` is **always** read.
- `appsettings.Development.json` is read **on top of it** only when the environment is `Development`, and its values win.
- In the same way, `appsettings.Production.json` is used in production.

4 Do I edit them?

Yes. You will add things like `ConnectionStrings` and JWT settings here.

> **Do not** put real passwords here, because the files are uploaded with the code.

INSIDE

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

- **Logging:** how many messages the app writes. `Information` shows general information. `Warning` shows only warnings (used for ASP.NET Core itself, to reduce noise).
- **AllowedHosts:** the host names allowed to reach the app. `*` means all.
- The Development file has no `AllowedHosts`, so that value comes from the base file. This is the "merge" idea.

> **Try it:** change `"Default": "Information"` to `"Warning"` in `appsettings.Development.json` and watch the messages in the terminal become fewer.

Properties/launchSettings.json

How the project is launched on your machine

1 What is it?

A file that defines how the project is launched on **your machine** during development.

2 Why does it exist?

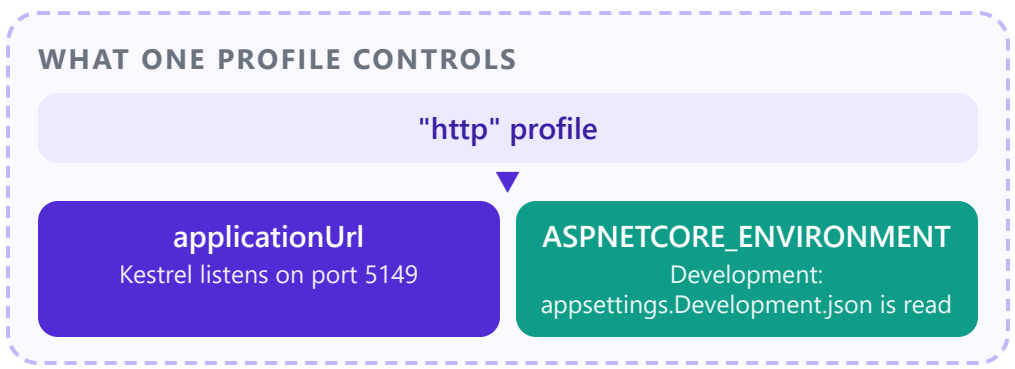
To set the port and the environment without typing them every time.

3 Who reads it, and when?

`dotnet run` and the editor (Visual Studio / VS Code) when you press Run. It is **not published** with the app, and the app does not read it in production.

4 Do I edit it?

Sometimes, for example to change the port.



INSIDE

```
"http": {
  "commandName": "Project",
  "launchBrowser": true,
  "applicationUrl": "http://localhost:5149",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
}
```

Setting	Meaning
`"http" / "https"`	A **profile** for each way of running. You choose one with `--launch-profile`.
`applicationUrl`	Address and port. The port is generated randomly when the project is created, so it differs between machines.
`launchBrowser`	Opens the browser when the app starts.
`ASPNETCORE_ENVIRONMENT`	The environment. It decides which `appsettings.*.json` is read, which links this file to the settings files.

The Dev Certificate

The HTTPS certificate for development

1 Why talk about it?

The `https` profile in `launchSettings.json` uses `https://localhost:7123`. HTTPS needs a **certificate**. Where does it come from on your machine?

2 What is it?

A **self-signed** HTTPS certificate created for your development machine only, so `https://localhost` works without buying a real certificate.

3 Why must I trust it?

The browser trusts only certificates issued by known authorities. A self-signed one is not, so the browser shows a **"Your connection is not private"** warning. The fix is to make your machine trust it.

4 Where does it live?

In your user **certificate store** on the machine. It is not part of the project, does not appear in its files, and is never uploaded to GitHub.

> The dev certificate is for **development only**. In production use a real certificate (for example Let's Encrypt, or one from your hosting provider). It is also what `UseHttpsRedirection` relies on when it redirects http to https.

COMMANDS

```
dotnet dev-certs https --check    # exists? trusted?
dotnet dev-certs https --trust    # create + trust
dotnet dev-certs https --clean    # delete it
```

- On Windows, `--trust` shows a dialog asking for approval. Click **Yes**.
- On Linux, `--trust` is not fully supported, and a manual setup depends on the distribution.

TRY IT

```
dotnet run --launch-profile https
```

- Open `https://localhost:7123` (use the port you have).
- If the certificate is not trusted, the browser shows a warning. Run `dotnet dev-certs https --trust` and reopen the browser: the warning is gone.

Run it and summary

What happens when the app starts, and who reads which file

RUN

```
dotnet run --launch-profile http
```

Open `http://localhost:5149` (use your port) and you see **Hello World!**

WHAT HAPPENED, FILE BY FILE

- `launchSettings.json` set the port and the environment.
- `.csproj` defined it as a web project.
- `Program.cs` built the app and registered the route.
- `appsettings` configured the logging.
- A **GET** request arrived on `/`, so `MapGet` ran and returned the text.

TRY IT YOURSELF

- Change the port in `launchSettings.json` and run the project on it.
- Add a new route `/about` that returns your name.
- Add `app.MapGet("/hello/{name}", (string name) => $"Hello {name}!");` and open `/hello/Ahmed`.

SUMMARY

File	What it is	Who reads it
`ProjectStructure.csproj`	The project's ID card	The `dotnet` tool at build time
`Program.cs`	The starting point	Runs when the app starts
`appsettings*.json`	The settings	The app at startup
`launchSettings.json`	Local launch settings	`dotnet run` and the editor only

> **Three stages in Program.cs:** configure the builder and register services, then `Build()`, then add middlewares and routes, then `Run()`.

